



CPU

CPU的组成

1. ALU
2. 寄存器
 - 通用寄存器
 - 控制寄存器
 - 地址寄存器AR
 - 数据寄存器DR
3. 总线
4. 控制器

控制器的组成

控制器的功能

控制机器各部件执行指令

1. 取指：根据PC从IMEM中取指令
2. 译码：分析指令要做什么操作，形成操作数地址
3. 执行指令：根据译码结果形成操作控制信号序列
4. I/O
5. 处理异常和请求
 - 中断：执行完当前指令后响应
 - DMA：完成当前机器周期后响应，让出总线，I/O设备与存储器进行传送数据操作

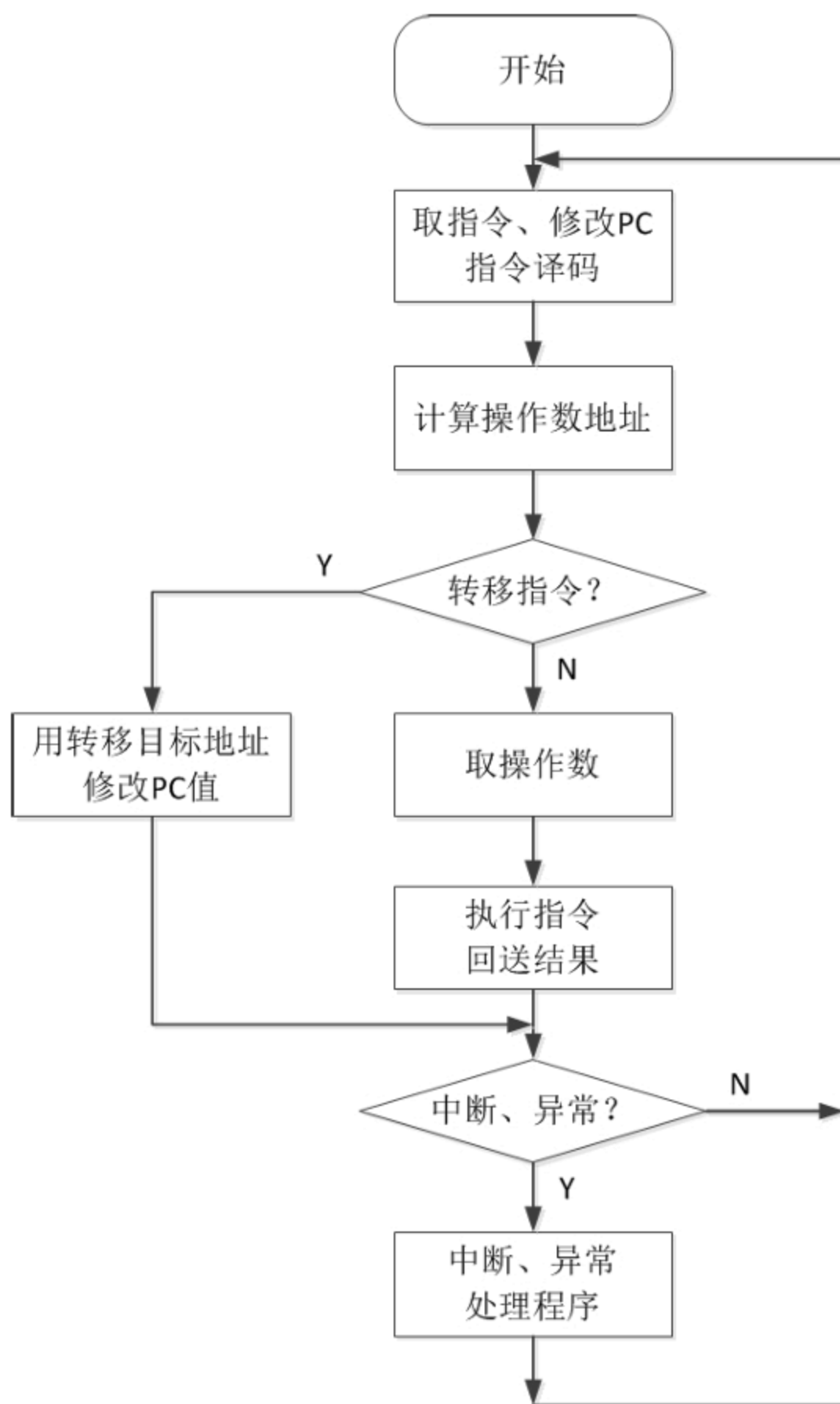
控制器的组成

1. 程序计数器PC
 - 存放即将要执行的下一条指令**地址**
 - 程序顺序执行，PC+1（4）
 - 跳转指令修改PC的值
2. 指令寄存器IR

- 存放当前正在执行的**指令**
- 3. 指令译码器
 - 对IR中的操作码进行分析
- 4. 脉冲源及启停线路
- 5. 时序控制信号形成部件

指令执行过程

1. PC送AR，从内存读入IR，修改PC
2. 译码器译码，得到操作和操作数地址
3. 取操作数（需要内存的话地址送AR，从内存读入）
4. 操作数送ALU并命令ALU进行操作
5. ALU结果送寄存器或内存
6. 如果是跳转指令则转移地址写入PC
7. 检查是否有中断，有转入处理程序，否则转回第一步



4个状态寄存器

- N (负数) : 结果为负置1
- Z (零) : 结果为零置1
- O (溢出) : 结果溢出置1
- C (进位) : 结果进位置1

一条加法指令的执行过程 (用数据总线DB、地址总线AB、控制总线CB)

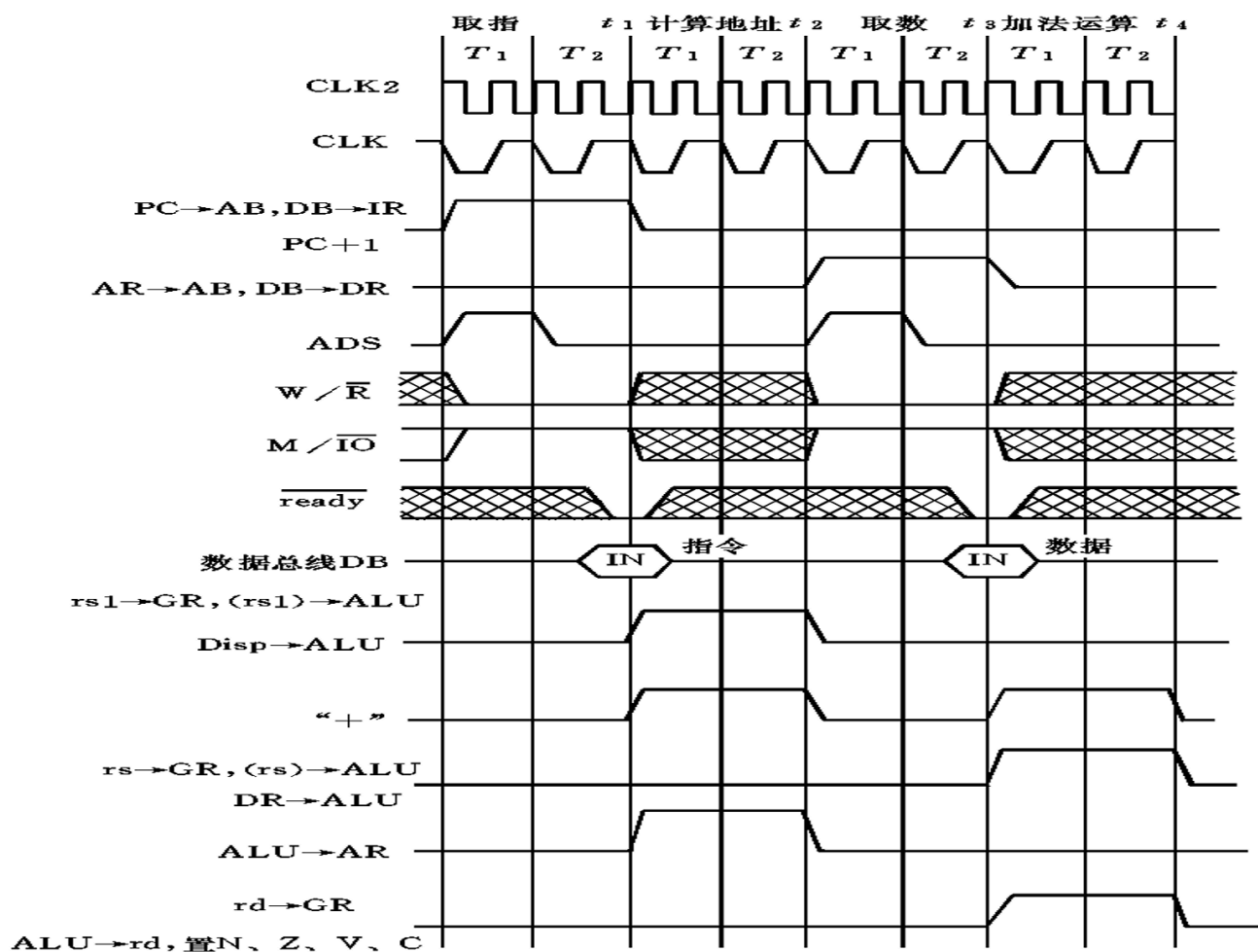
假设指令格式:

D31	D24	D23-D20	D19-D16	D15	D0
操作码	rs,rd	rs1	Imm(disg)		

ADD AL, SI[1000H]; $AL \leftarrow AL + [SI + 1000H]$

D31	D24	D23-D20	D19-D16	D15	D0
ADD	AL	SI	1000H		

1. $PC \rightarrow AB$, $W//R=0$, $M//IO=1$, $DB \rightarrow IR$, $PC+1$
2. $rs1 \rightarrow GR$, $(rs1) \rightarrow ALU$, $disp \rightarrow ALU$, “+”, $ALU \rightarrow AR$
3. $AR \rightarrow AB$, $W//R=0$, $M//IO=1$, $DB \rightarrow DR$
4. $rs \rightarrow GR$, $(rs) \rightarrow ALU$, $DR \rightarrow ALU$, “+”, $ALU \rightarrow GR(rd)$, 置NZOC状态位



控制器的控制方式

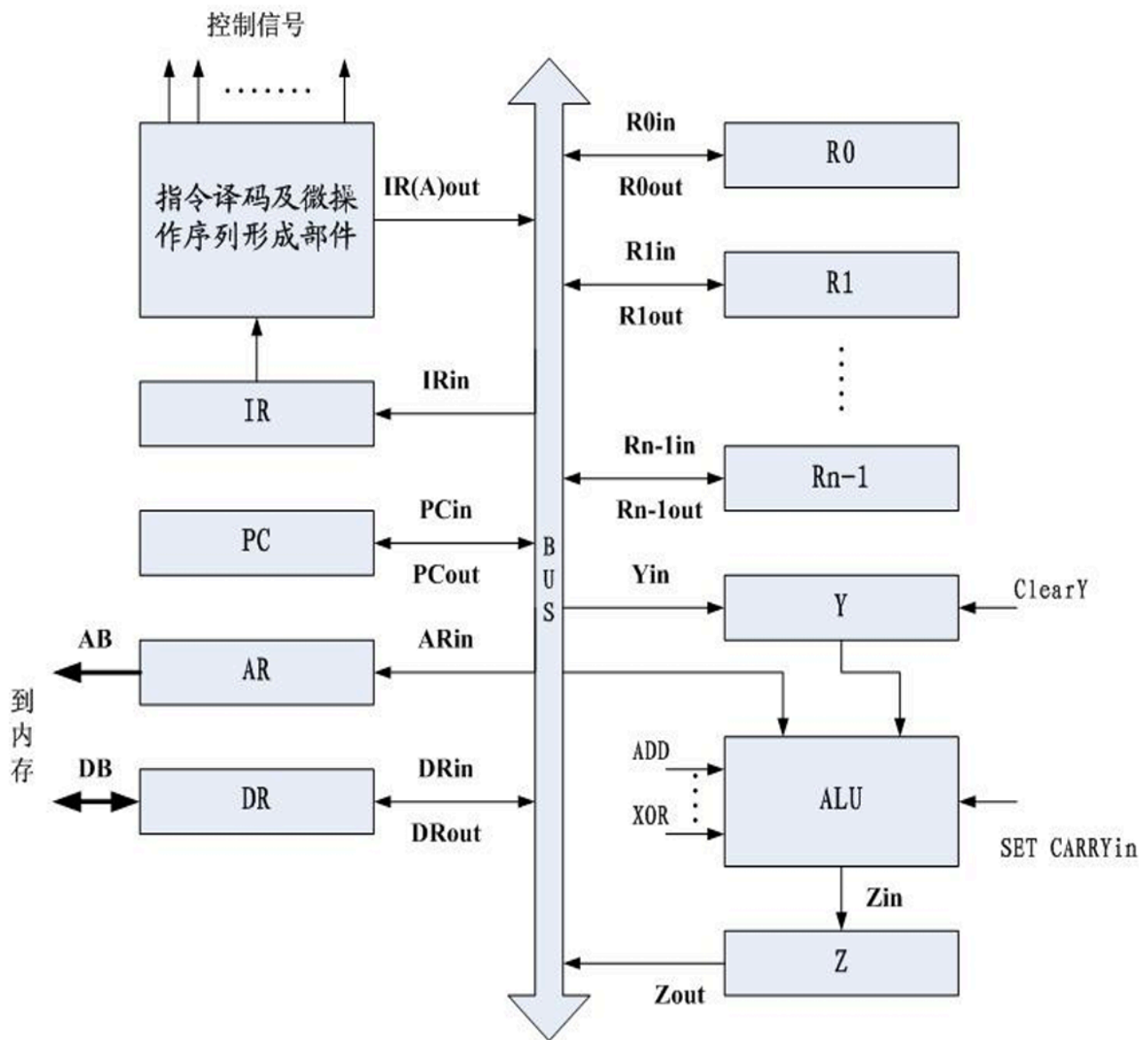
1. 同步控制：每条指令都是在相同的时间完成
2. 异步控制：根据前一条指令是否已完成的“完成”信号来控制
3. 联合控制：对大多数的指令采用同步控制方式，对个别少数无法采用同步控制方式的指令，采用区别对待的办法，在时间上给予延长

控制器的组成方式

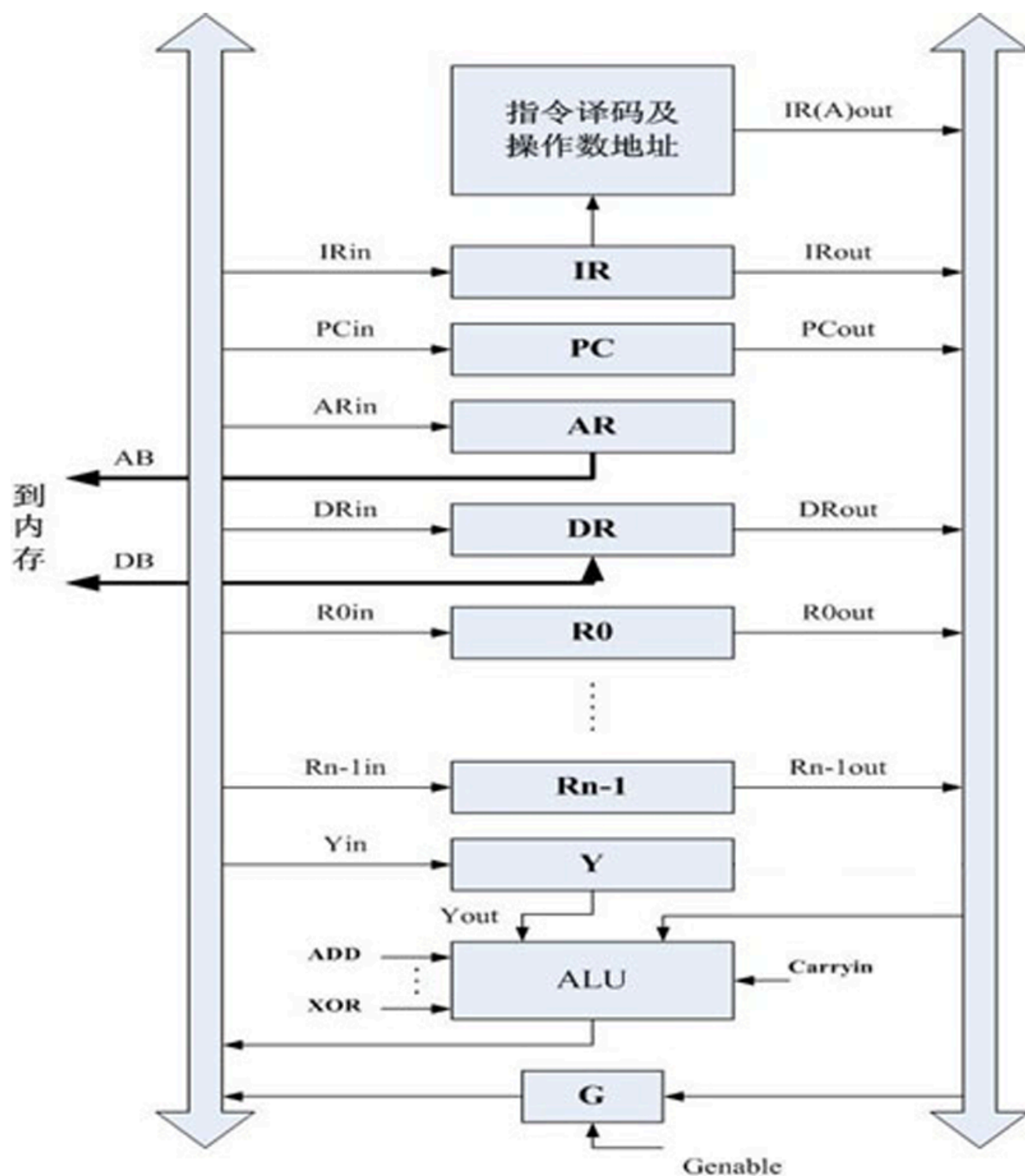
1. 组合逻辑：以逻辑代数为工具进行设计，由门电路构成
2. 微程序：用微程序设计方法设计，由微程序存储器和门电路构成

数据通路的构成

1. 单总线：同一时刻，可以有多个部件接收数据，但只能一个部件发送数据



2. 双总线：in总线负责发送数据给部件，out总线负责从部件接收数据，out中数据同步给in需要控制信号



3. 基于专用通路结构：自己画数据通路

4. 单周期/多周期处理器

- 单周期处理器是指所有的指令在一个时钟周期内完成的处理器，尽管不同指令执行时间不同，但对单周期处理器而言，时钟周期必须设计成对所有指令都等长

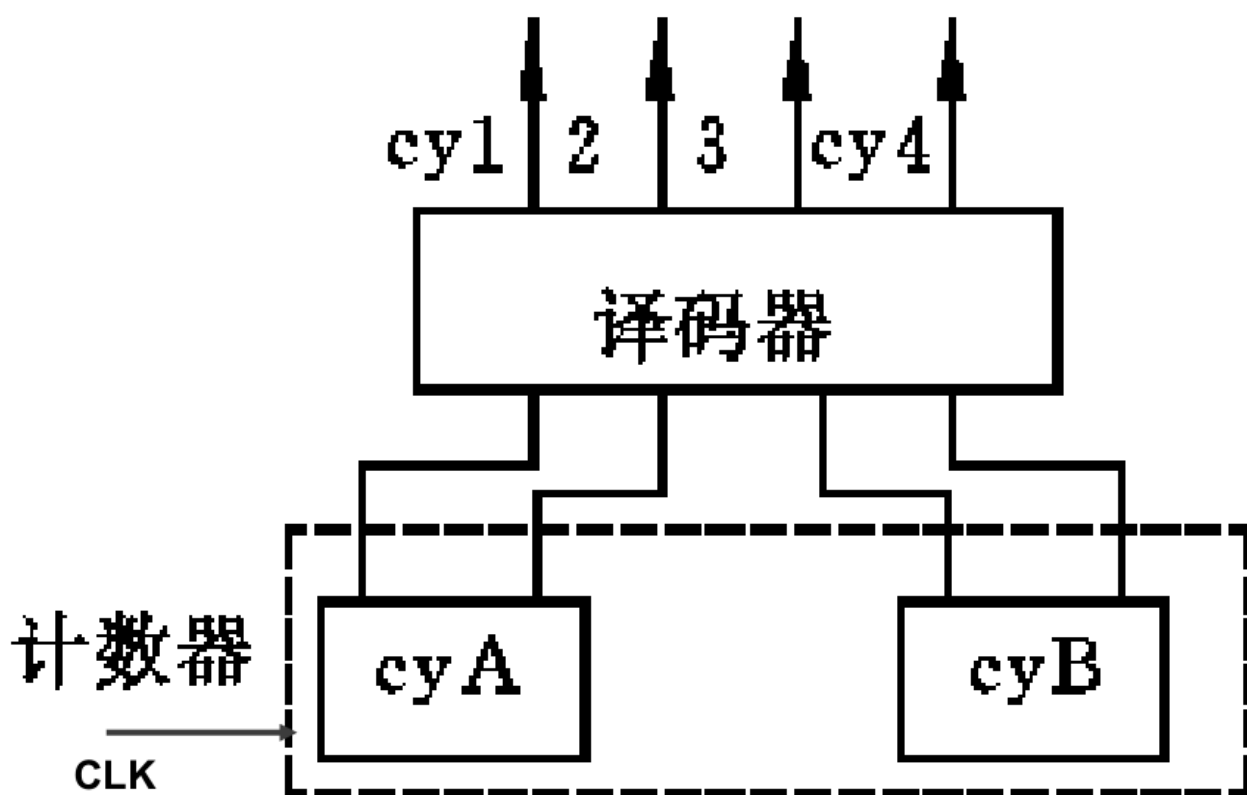
- 多周期处理器是指指令可根据执行的时间长度，在一个时钟周期或多个时钟周期内完成的处理器，所以，对不同指令执行时间，对多周期处理器而言，指令所用的时钟周期数也不同
- 单周期处理器中资源（如加法器）无法重复使用，而多周期可以

组合逻辑控制器

时序与节拍

一条指令的执行分为：取指、计算地址、取数、执行等

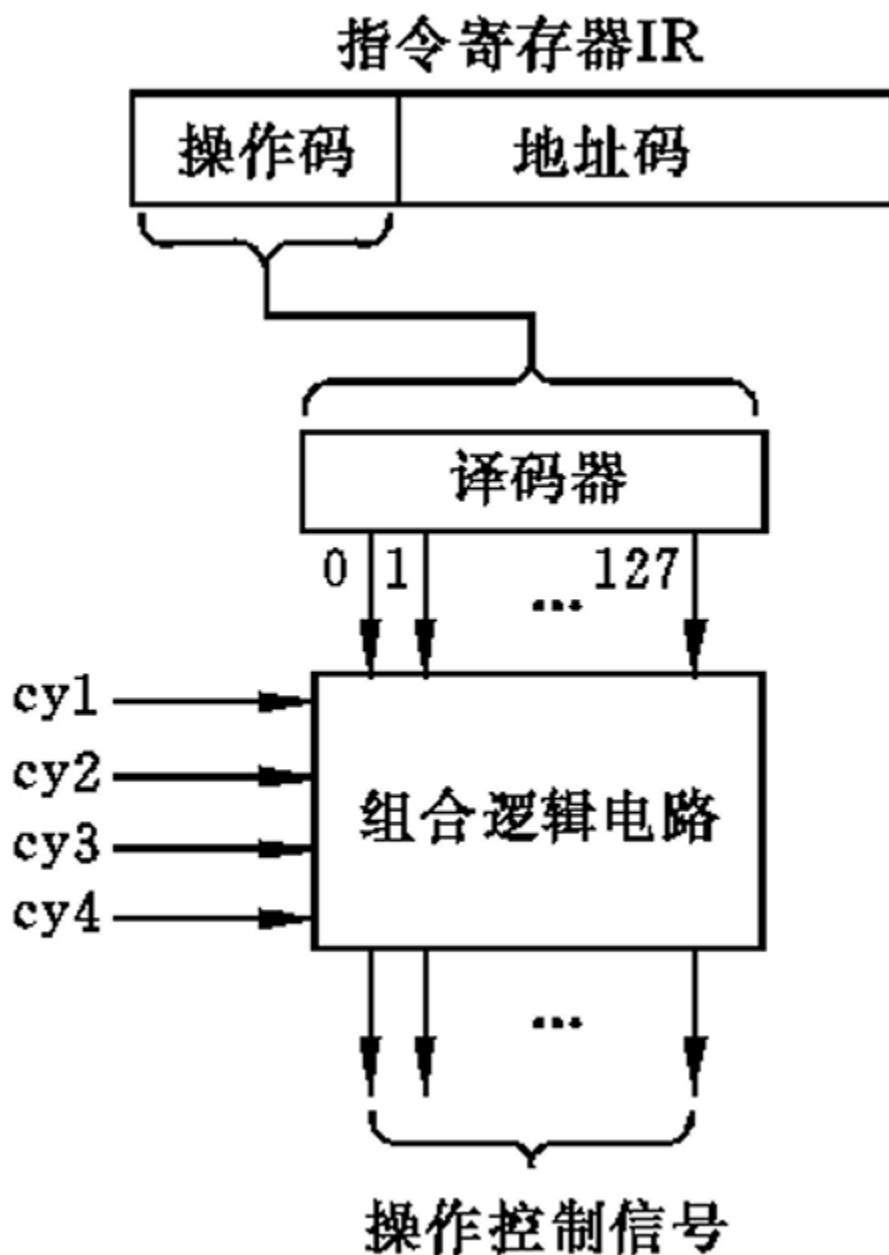
- 对于多周期处理器，每一步用一个机器周期实现
 - 指令周期由若干个机器周期构成
 - 机器周期由若干个时钟周期构成
- 对于单周期处理器，所有操作都在一个机器周期中实现



操作控制信号的产生

- 操作码译码器：输出反映当前正在执行的指令

- 由译码器的输出和cy1~4作为输入，使用逻辑电路产生操作控制信号



组合逻辑控制器设计 (CPU31做过)

1. 根据指令功能设计数据通路
2. 绘制指令流程图
3. 编排指令操作时间表
4. 进行微操作综合
5. 构成微操作序列形成部件的组合逻辑网络

微程序控制器

把操作控制信号编成“微指令”，存放在一个只读存储器中，机器执行指令时，就从只读存储器中一条一条地读出这些微指令，从而产生所需的各个控制信号

基本概念

- 微操作：一条指令的功能是通过按一定次序执行一系列基本操作完成的，这些基本操作称为微操作
- 微指令：同时发出的控制信号所执行的一组微操作称为微指令
- 微程序：每条指令的功能均由微指令序列解释完成，这些微指令序列的集合就叫做微程序
- 控制存储器：存放控制命令（信号）和下一条执行的微指令地址（下址）
 - 每一条微指令（可能）对应的下址都是固定的，因此可以做成只读存储器

微指令字的编码设计

1. 直接控制法
 - 在控制字段，每一位代表一个微命令
 - 设计微指令时，发出某个微命令则将对应该位置1
 - 在微命令多时微指令字长达到难以接受的地步
2. 字段直接编译法
 - 选出互斥的微命令编成一组，成为微指令字的一个字段，为该字段增加一个译码器即可
 - 相容性：把可以在同一时刻发出的分在不同组
 - 相斥性：把不能在同一时刻发出的分在同一组
3. 字段间接编译法
 - 在字段直接编译法的基础上还规定一个字段的某些微命令，要兼由另一字段中的某些微命令来解释
 - 减少了指令长度
 - 但可能削弱并行控制能力
4. 常数源字段
 - 类似于直接操作数
 - 用来给某些部件发送常数

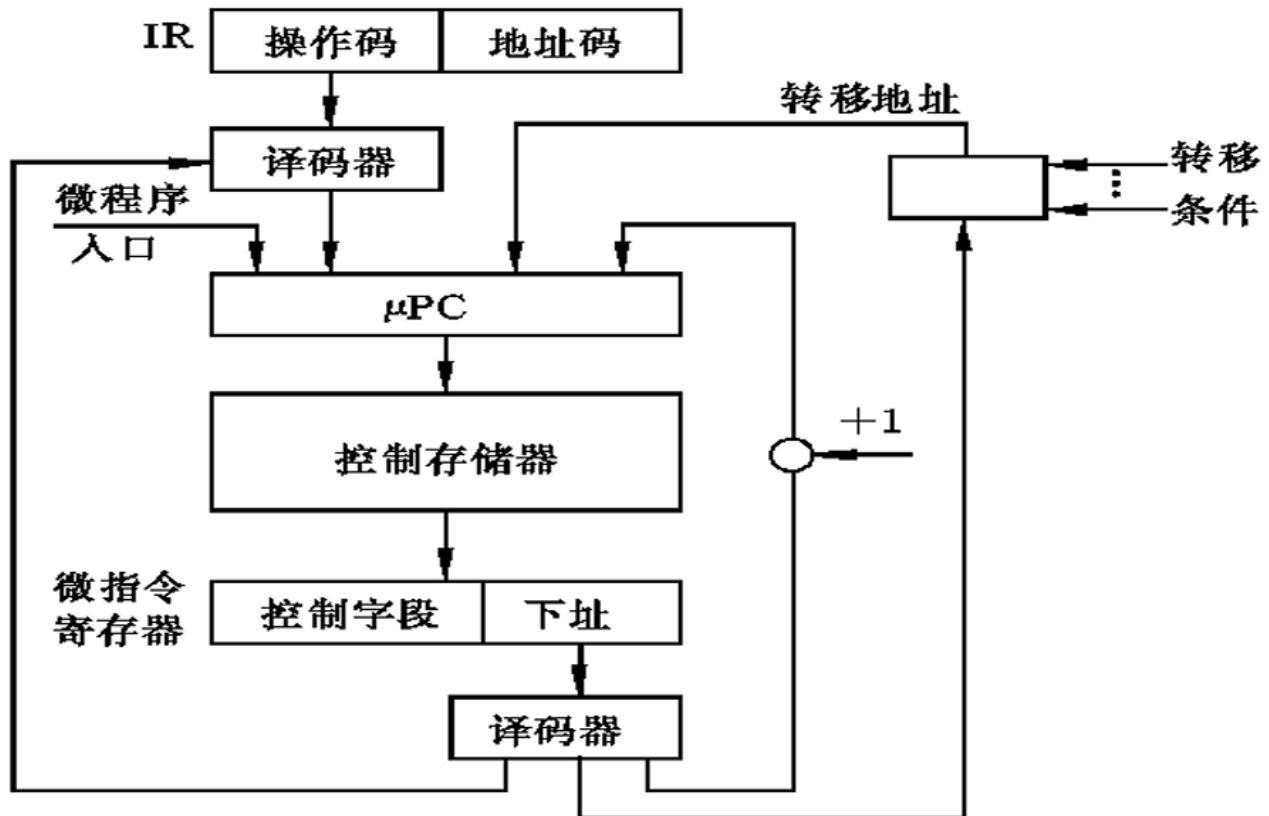
微程序流的控制

研究当前微指令执行完毕后，怎样控制产生后继微指令的微地址

产生后继微指令地址的方法

1. 增量方式

- 顺序执行：下址=现址+1
- 跳转执行：下址=转移地址



2. 增量与下址字段结合

- 下址字段分为两部分：转移控制字段BCF和转移地址字段BAF
- 顺序执行：下址=现址+1
- 跳转执行：下址=BAF

BCF 字段		硬件条件	计数器CT		返回寄存器RR输入	后继微地址
编码	微命令名称		操作前	操作		
0	顺序执行	×	×	×	×	$\mu\text{PC}+1$
1	结果为0转移	结果为0	×	×	×	BAF
		结果不为0				$\mu\text{PC}+1$
2	结果溢出转移	溢出	×	×	×	BAF
		不溢出				$\mu\text{PC}+1$
3	无条件转移	×	×	×	×	BAF
4	测试循环	×	为0	CT-1	×	$\mu\text{PC}+1$
			不为0			BAF
5	转微子程序	×	×	×	$\mu\text{PC}+1$	BAF
6	返回	×	×	×	×	RR
7	操作码形成微址	×	×	×	×	由操作码形成

- 微指令执行效率高，速度快
- 缺点：微指令字长比较长，增加了ROM横向容量，编制微程序需要非常熟悉机器的数据通路，应用困难

垂直型微指令

- 设置有微操作码字段
- 采用微操作码编译法，由微操作码规定微指令的功能
- 采用短指令格式，一条微指令只能产生一、二个控制信号，并行控制能力低
- 微指令的各二进制位与数据通路的各控制点之间完全不存在直接的对应关系
- 在微指令中需要给出目的部件或源部件的编址
- 缺点：横向信息位比较短，为解释一条机器指令或某种处理过程需要的微指令数多，因而编制出的微程序较长，要求ROM容量大，执行速度慢

水平型和垂直型微指令的比较

1. 水平型微指令并行操作能力强，效率高，灵活性强，垂直型微指令则差
2. 水平型微指令执行一条指令的时间短，垂直型微指令执行时间长
3. 由水平型微指令解释指令的微程序，具有微指令字比较长，但微程序短的特点。垂直型微指令则相反，微指令字比较短而微程序长
4. 水平型微指令用户难以掌握，而垂直型微指令与指令比较相似，相对来说，比较容易掌握
5. 本质区别：水平型微指令面向处理机内部控制逻辑的描述，垂直型微指令面向算法描述

毫微程序设计

- 用以解释微程序的一种微程序
- 第一层采用垂直微程序，第二层采用水平微程序
- 两个控制存储器容量都不大， μ CS横向字长较短，nCS纵向容量较小
- 由于垂直微指令是面向算法描述的，用户编制微程序较容易
- 水平的毫微指令能产生较多的并行操作，控制效率高
- 缺点：要读两个存储器，速度较慢

微指令的执行

对于取指、执行，有串行、并行两种执行方式

- T_{cs} ：控制存储器的存取时间

- T_{cpu} : 微指令的执行时间, 相当于CPU的一个节拍时间, 一般是 T_{cs} 的3倍
- T_{μ} : 执行一条微指令的时间 (称微周期)

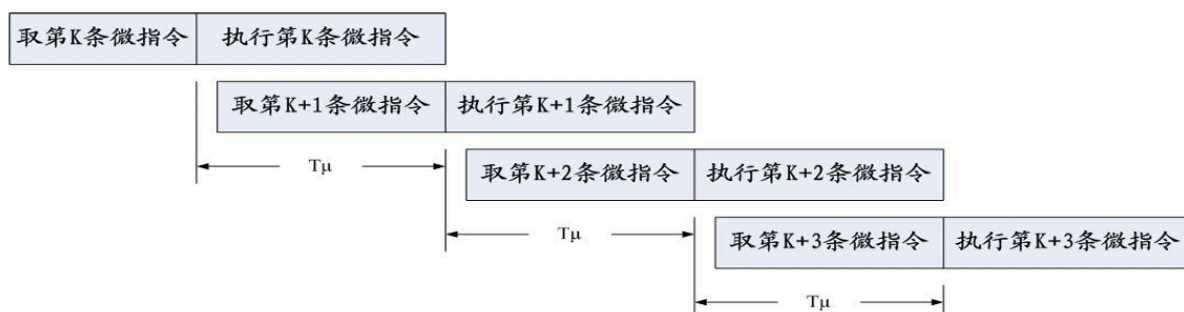
1. 串行执行: 取微指令和执行微指令按顺序进行

- 缺点: 执行速度慢
- 优点: 微程序控制器结构简单

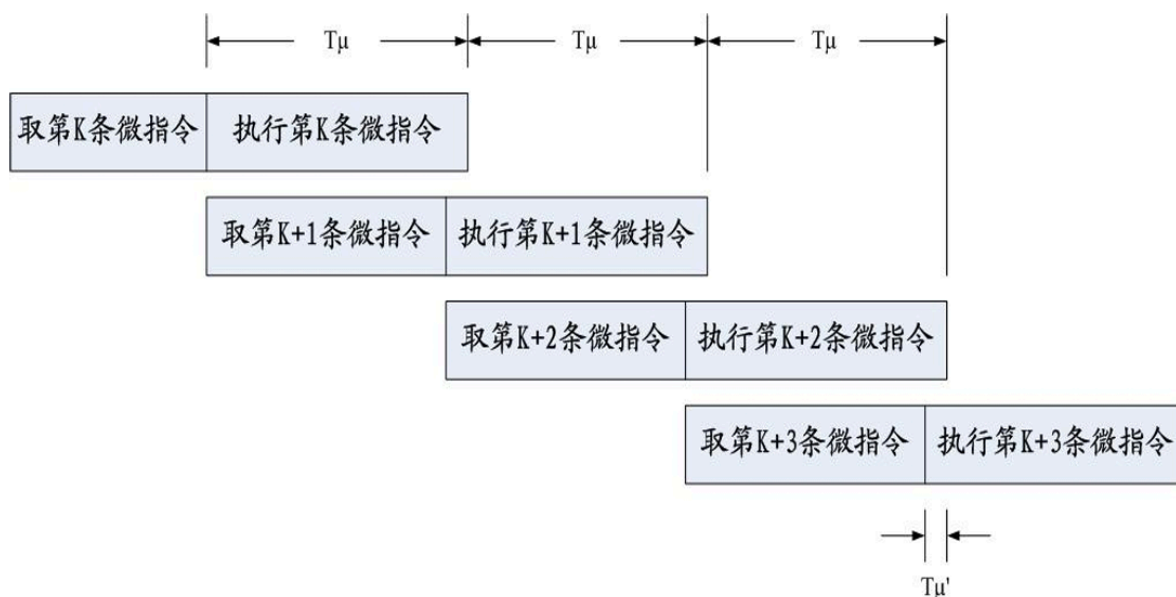


2. 并行执行: 在本条微指令执行结束前, 下一条微指令提前从控存取出, 称为流水执行

- 微周期不重叠: 每条微指令的执行和预取下条微指令的操作都能在一个微周期内完成



- 微周期重叠: 每条微指令的操作, 在本微周期内不能完成, 还借助于下一个微周期的时间里继续进行



3. 并行执行的转移方式

- 退回到串行执行

- 对于转移微指令，改为不立即转移
- 安排一条专门实现转移的微指令
- 这样下一条微指令的地址就是确定的了
- 当转移微指令多的时候时间损失大
- 还增加了控制存储器容量

ii. 猜测法

- 设计者事先猜测一条可能性最大的微指令作为下一条
- 命中则继续执行，不命中则废弃已读的微指令，重新取出正确的再执行

iii. 并行读出多条微指令

- 将控存分为多个体或者采用单体多字结构，使一次访问能读出多条微指令
- 在转移点同时读出多条后续可能的微指令
- 运算结果判别后选取其中一条
- 损失时间最小
- 但硬件复杂性增加

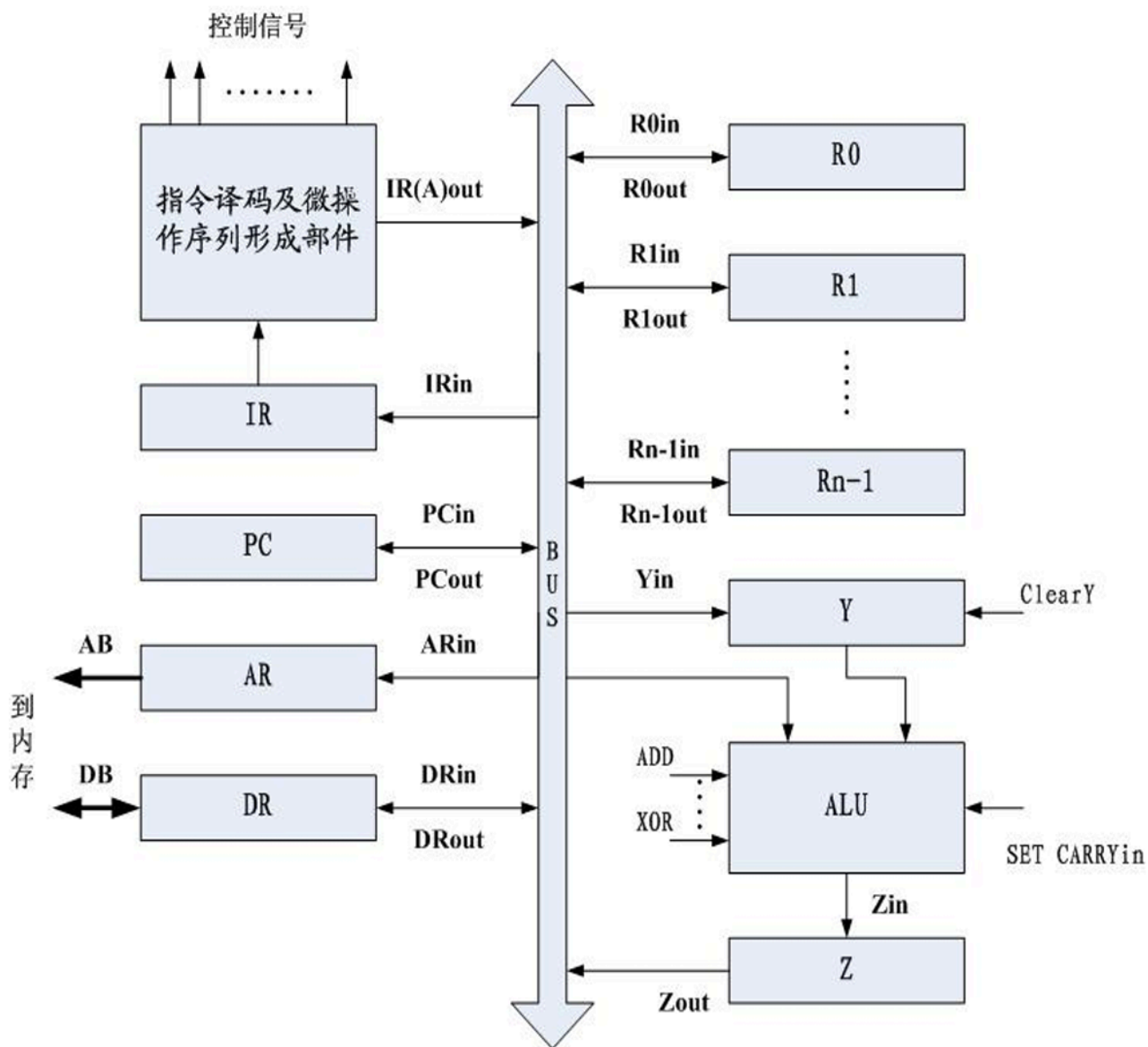
动态微程序设计

在可改写控存的支持下，能根据不同应用目标需要，随时改变或切换相应的微程序

1. 人工变更：费时间，效率低
2. 自动变更：依靠操作系统的管理和控制自动实现

指令的执行

单总线结构



1. ADD Loca,R1;(Loca)+R1→R1
2. JMP D;PC+D→PC
3. JNZ D;Z=0:PC+D→PC,Z!=0:PC+1→PC
4. MOV R1,R2;(R1)→R2
5. XOR (R3),(R4);((R3))XOR((R4))→(R4)

ADD微程序如下:

- (1) PCout,ARin,Read,
ClearY,Set Carryin,
Add,Zin**
 - (2) Zout,PCin,Wait
MFC**
 - (3) DRout,IRin**
 - (4) IR (A) out,ARin,
Read**
 - (5) R1out,Yin,
Wait MFC**
 - (6) DRout,Add,Zin**
 - (7) Zout,R1in**
 - (8) End**
-

JMP微程序如下:

- (1) PCout,ARin,Read,
ClearY,Set Carryin,
Add,Zin**
- (2) Zout,PCin,Wait
MFC**
- (3) DRout,IRin**
- (4) PCout,Yin**
- (5) IR(A)out,Add,Zin**
- (6) Zout,PCin**
- (7) End**

JNZ微程序如下:

- (1) PCout,ARin,Read,
ClearY,Set Carryin,
Add,Zin**
- (2) Zout,PCin,Wait
MFC**
- (3) DRout,IRin**
- (4) IF Z=1 THEN End**
- (5) PCout,Yin**
- (6) IR(A)out,Add,Zin**
- (7) Zout,PCin**
- (8) End**

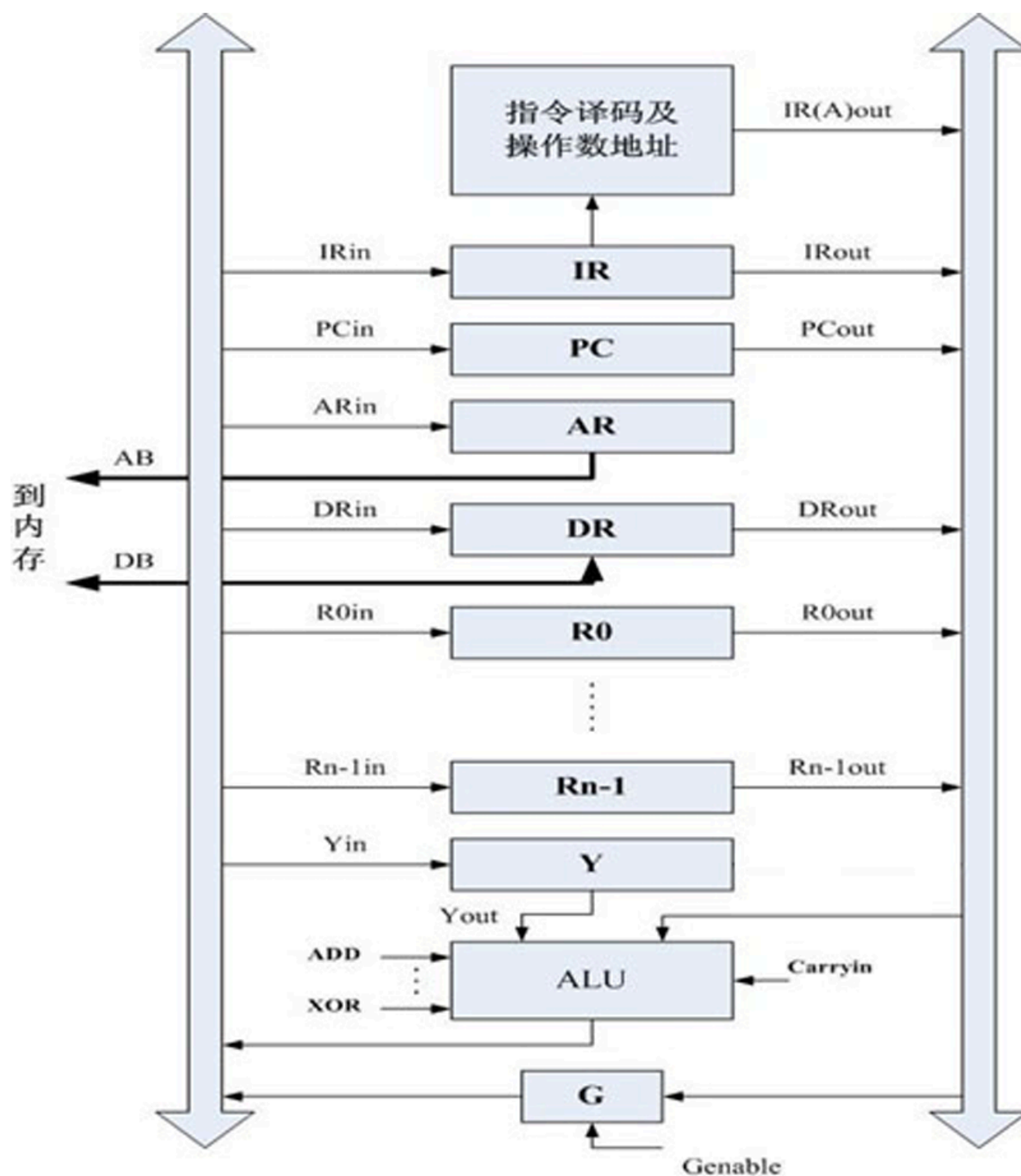
MOV微程序如下:

- (1) PCout,ARin,Read,
ClearY,Set Carryin,
Add,Zin**
- (2) Zout,PCin,Wait
MFC**
- (3) DRout,IRin**
- (4) R1out,R2in**
- (5) End**

XOR微程序如下:

- (1) PCout,ARin,Read,
ClearY,Set Carryin,
Add,Zin**
- (2) Zout,PCin,Wait MFC**
- (3) DRout,IRin**
- (4) R3out,ARin,Read,
Wait MFC**
- (5) DRout,Yin**
- (6) R4out,ARin,Read,
Wait MFC**
- (7) DRout,Xor,Zin**
- (8) Zout,DRin,Write,
Wait MFC**
- (9) End**

双总线结构



1. ADD Loca,R1;(Loca)+R1→R1
2. JZ D;Z=1:PC+D→PC,Z!=1:PC+1→PC
3. MOV (R1),R2;((R1))→R2

ADD微程序如下:

- (1) PCout,Genable,ARin,
Read,ClearY**
- (2) PCout,Set Carryin,Add,
PCin,Wait MFC**
- (3) DRout, Genable,IRin**
- (4) IR (A) out, Genable,
ARin,Read**
- (5) R1out, Genable,Yin,
Wait MFC**
- (6) DRout,Add,R1in**
- (7) End**

JZ微程序如下:

- (1) PCout,Genable,ARin,
Read,ClearY**
- (2) PCout,Set Carryin,Add,
PCin,Wait MFC**
- (3) DRout, Genable,IRin**
- (4) IF Z=0 THEN End**
- (5) PCout,Genable,Yin**
- (6) IR(A)out,Add,PCin**
- (7) End**

MOV微程序如下:

- (1) PCout,Genable,ARin,
Read,ClearY
- (2) PCout,Set Carryin,Add,
PCin,Wait MFC
- (3) DRout, Genable,IRin
- (4) R1out, Genable,ARin,
Read,Wait MFC
- (5) DRout, Genable,R2in
- (6) End

流水线工作原理

- 假如我们把上条指令的执行和取下条指令的操作，在时间上重叠起来进行，将大幅度提高程序的执行速度
- 如将一条指令的取指到执行分成4段，除第一条指令外，以后每 $1/4T$ 就完成一条指令的执行，效率提高4倍
- 在时间划分上，最好每段长度基本上一致
- 顺序控制：
 - 顺序流动，即流入的顺序与流出的顺序相同
 - 异步流动，即流入的顺序与流出的顺序不一致
- 流水线阻塞：后一条指令需要用到前一条指令的结果，称为数据相关
 - 解决办法：直接到数据处理部件取数据
 - 可以提高效率，但控制变得复杂
- 程序转移的影响：遇到转移指令无法确定下一条指令
 - 解决办法：同微指令

- 中断的影响：
 - 不精确断点法：对中断时还未进入流水线的后续指令不允许其再进入，但已在流水线中的所有指令则仍执行完毕，然后转入中断处理程序
 - 精确断点法：在当前指令执行完后，进入中断处理程序，对在流水线中未执行完的指令，将其所有状态保存起来，等中断程序执行完后，恢复所有状态再执行下去